

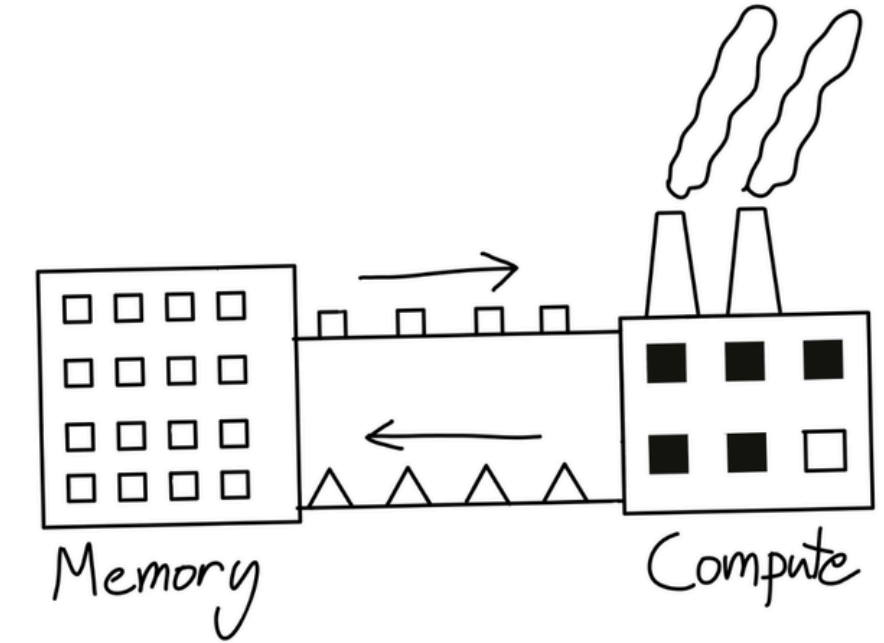
Background: Deep Learning과 Memory

딥러닝(DNN) 연산이란?

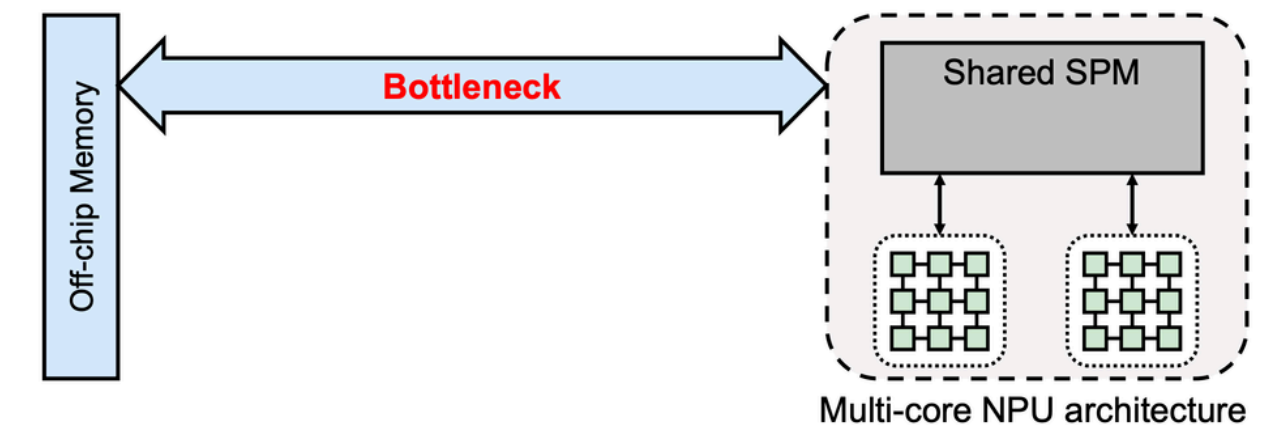
- 연산의 관점에서 DNN은 input tensor를 layer마다 읽고 연산한 후 output tensor로 쓰는 계산 그래프라고 볼 수 있다
 - 즉 1. 대량의 tensor를 읽고 쓰는 **메모리 이동**과 2. 읽은 tensor를 바탕으로 연산(ex. 행렬곱)하는 것으로 나눌 수 있다

메모리 이동이 병목(memory-bound)이 될 수 있는 경우

- compute-bound and memory-bound
 - 실제 데이터를 연산하는 데 시간이 많이 쓰이는가 vs 연산을 위해 DRAM에서 SRAM으로 메모리를 이동하는 시간이 더 오래 걸리는가
- arithmetic intensity: workload가 byte(memory)당 얼마나 많은 연산을 하는가
- 딥러닝의 모든 경우가 memory-bound한 것은 아니다. 그러나...
 - 일반적으로 memory unit은 compute unit에 비해 성능을 높이기 더 어렵고 비싸다
 - 따라서 workload의 연산량이 충분히 높지 않다면 메모리 이동이 주요 병목이 될 수 있다
 - 대표적인 예시: Embedding, Normalization, LLM Decode 등.
 - 이들의 공통적인 특징은 낮은 arithmetic intensity이다
 - 즉 읽고 쓰는 input tensor 양에 비해 각 byte에 대한 연산량이 작다는 점이다



https://horace.io/brrr_intro.html



https://seonjinna.github.io/assets/pdf/micro23_slides.pdf

메모리 병목을 줄이기 위한 NPU 최적화

Main Idea of NPU optimization

- 앞서 살펴보았듯 딥러닝 workload 중 일부는 낮은 arithmetic intensity를 가지고 이는 memory-bound하게 될 수 있다
- 이 경우 compute unit (ATOM NPU의 경우 Neural Engine)을 충분히 활용하지 못하기 때문에 낮은 성능으로 이어진다

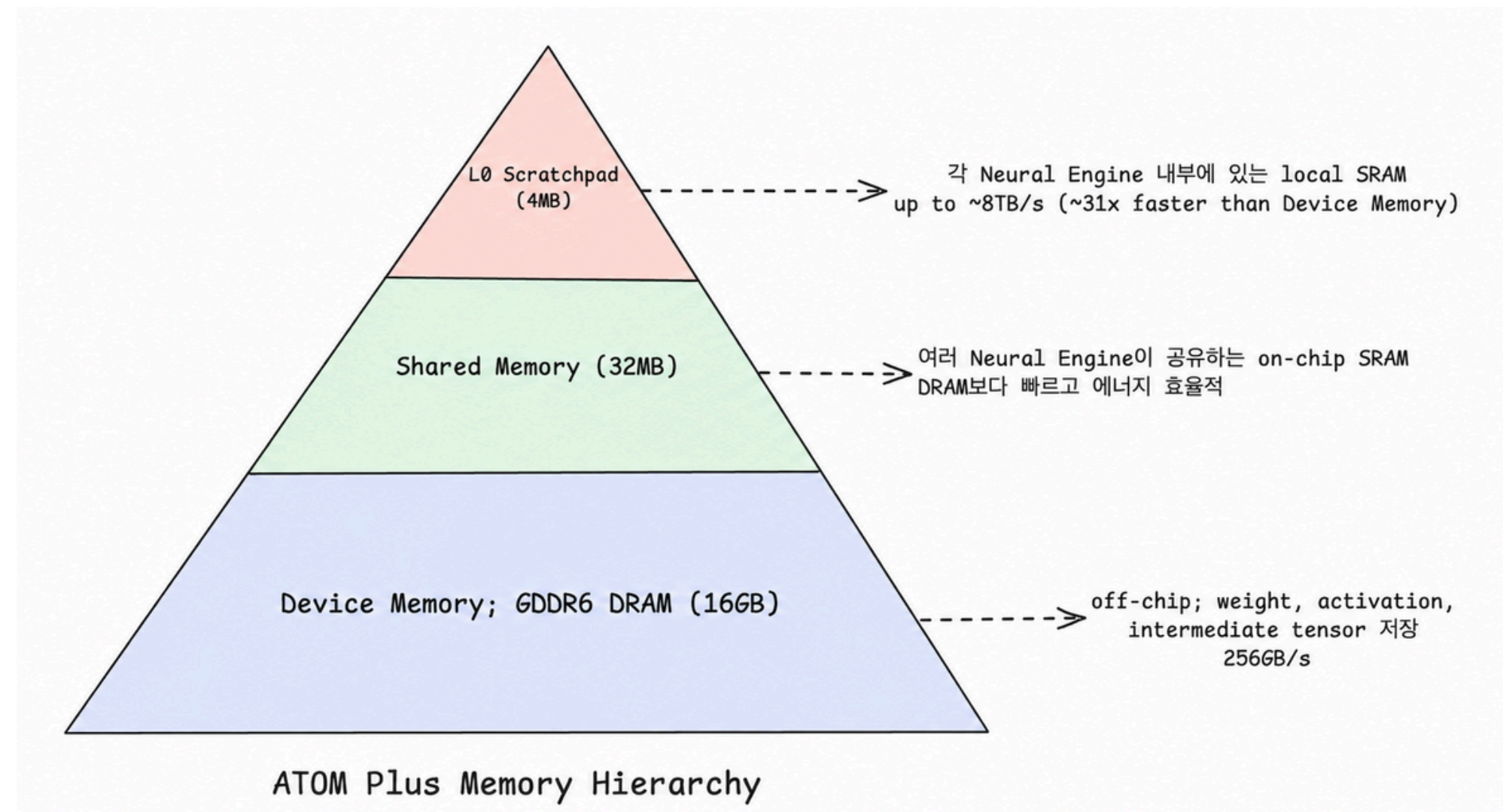
NPU에서는 이러한 메모리 병목을 줄이기 위해 크게 두 가지 접근 방식을 사용한다:

Idea 1. SRAM에 한 번 올린 데이터를 최대한 재사용해서 DRAM 접근 횟수를 최소화한다.

Idea 2. DRAM 데이터 이동으로 인해 연산이 대기하는 시간을 줄인다

참고) ATOM Plus Memory 계층 구조

- 뒤의 내용을 이해하기 위해 간단히 ATOM Plus memory 계층 구조를 확인해보면 다음과 같다:
 - L0 Scratchpad: 4MB의 on-chip SRAM 메모리
 - Shared Memory: 32MB의 on-chip SRAM 메모리
 - Off-chip GDDR6 DRAM: 16GB의 off-chip DRAM 메모리
 - DRAM 메모리에서의 데이터 이동은 Scratchpad에 비해 약 ~31배 느리기 때문에 SRAM을 최대한 잘 활용해야 된다



Idea 1. SRAM에 한 번 올린 데이터는 최대한 재사용해서 DRAM 접근 횟수를 최소화한다.

Idea 1

- SRAM을 최대한 활용하기 위해서는 한 번 올린 tile을 여러 **MAC* 연산**에서 재사용해 DRAM traffic을 줄여야 한다
 - *MAC(Multiply-Accumulate) 연산: activation과 weight를 곱한 뒤 그 결과를 partial sum에 더하는 연산.

대표적인 Compiler 최적화 방법

1. Tiling

- SRAM 용량 한계(수십 MB)로 큰 tensor는 tile 단위로 나눠 연산한다
- tile 단위로 처리하면 해당 tile 내 중간 결과를 DRAM에 쓰지 않고 SRAM에서 계속 재사용할 수 있다

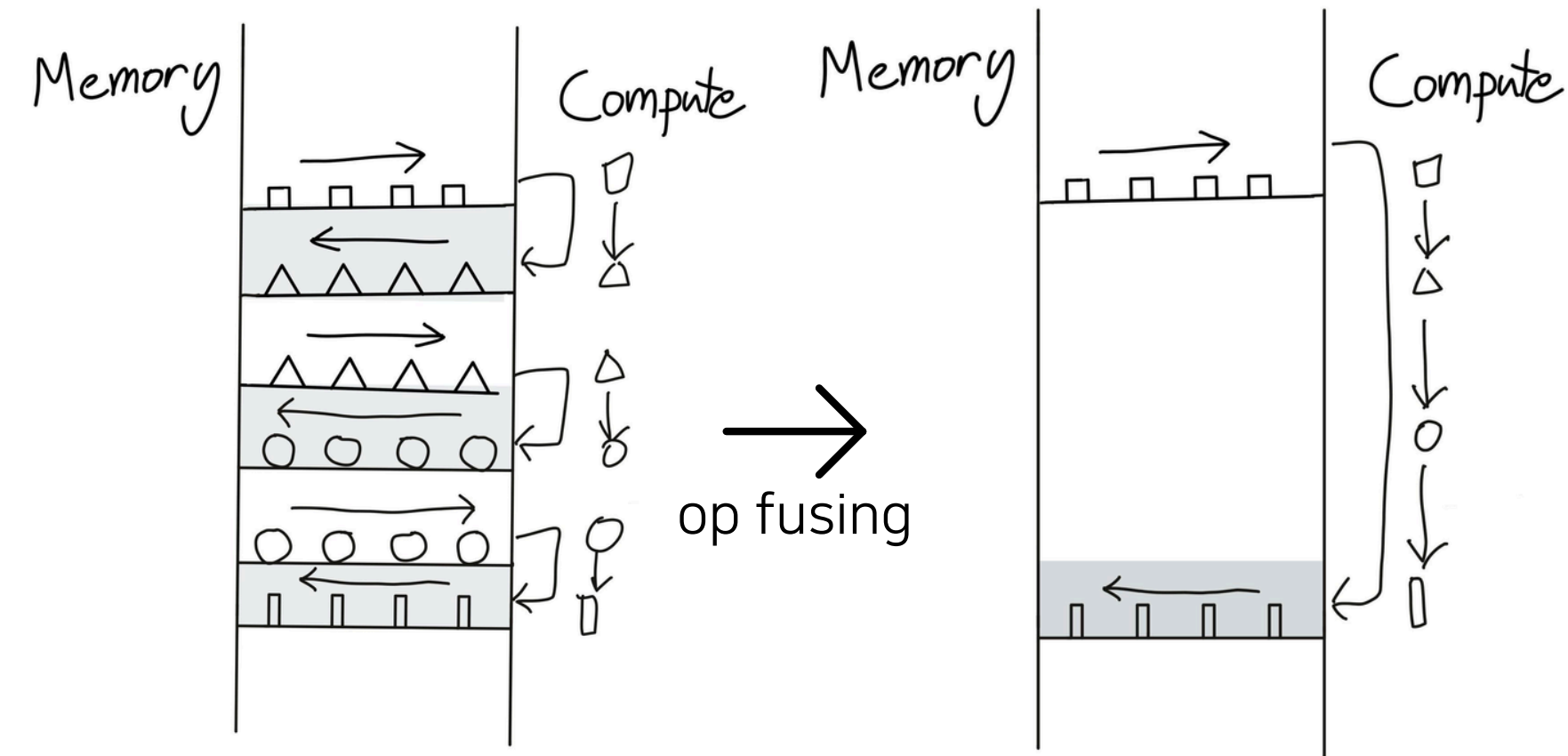
2. Op Fusing

- 여러 연산을 하나의 연산으로 합치는 최적화 방식
- 중간 activation 결과를 DRAM에 쓰지 않고 SRAM에 그대로 두고 다음 연산에서 사용할 수 있게 된다 (Ex. Matmul 다음 Activation을 하나의 연산으로 합침)

3. Buffer Reuse

- 한 tile의 연산이 끝난 후 SRAM 공간을 즉시 다음 tile에서 재사용함으로써 SRAM을 더 효율적으로 활용할 수 있다

→ 이러한 최적화들을 통해 SRAM에 올린 데이터를 최대한 오래 유지하면서 불필요한 DRAM 접근을 줄일 수 있다.



https://horace.io/brrr_intro.html

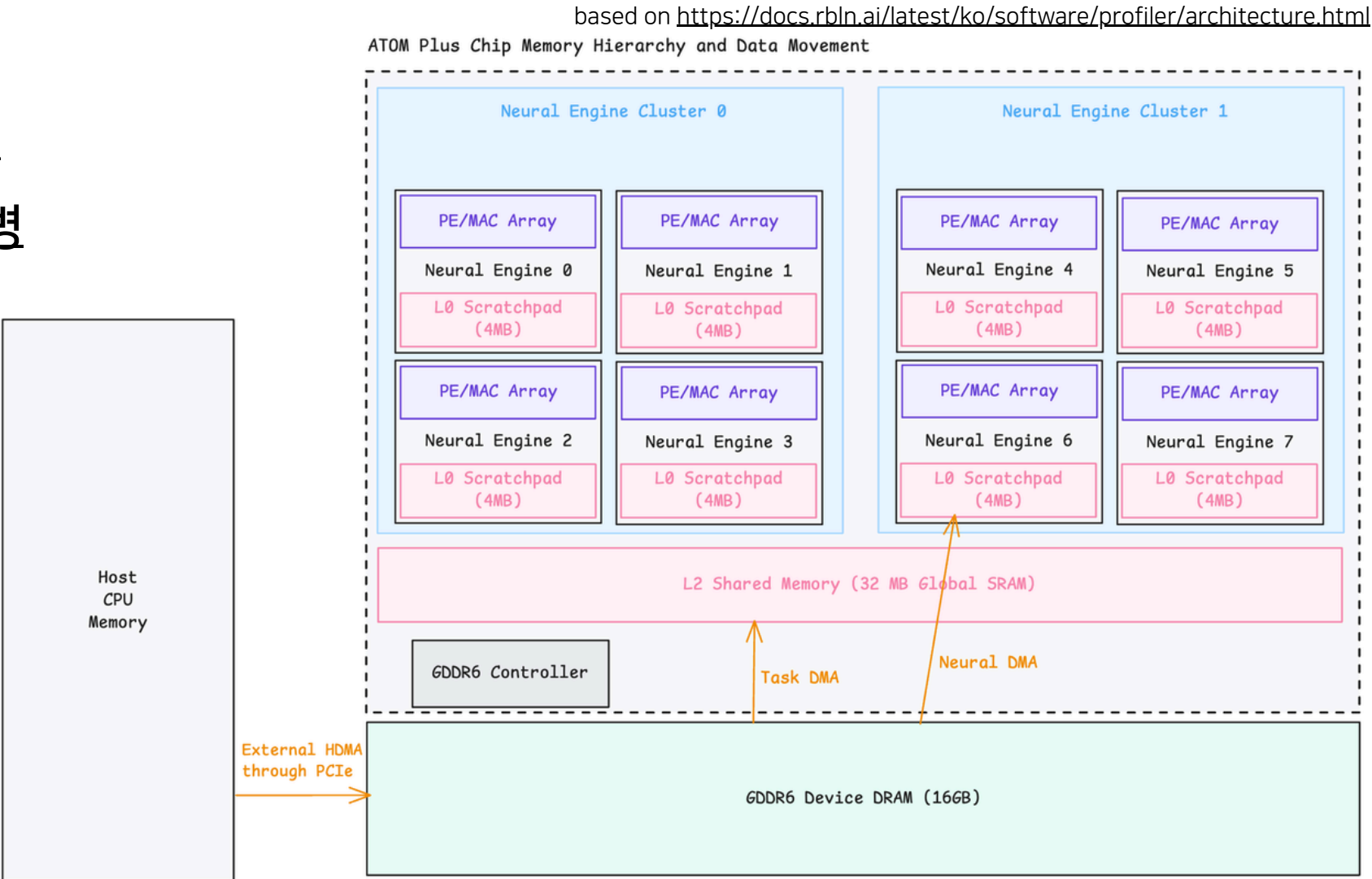
Idea 2. DRAM 데이터 이동으로 인해 연산이 대기하는 시간을 줄인다

Idea 2

- 연산이 진행되는 도중에 다음 연산에 쓸 tensor data를 미리 load하도록 겹치는 방식으로 compute unit이 DRAM 데이터를 기다리는 시간(memory bound 병목)을 줄인다

이를 안전하게 수행하기 위해 tensor 연산을 데이터 이동 명령어(data movement command)와 연산 명령어(compute command)로 쪼갬 후 적절한 의존성을 설정한다

- 데이터 이동 명령어
 - DMA*를 통해 필요한 input tensor를 Device DRAM에서 SRAM 메모리로 옮긴다
 - *DMA: compute unit이 직접 데이터를 복사하지 않고 별도의 데이터 이동 장치가 DRAM의 tensor를 on-chip SRAM/Scratchpad로 옮기는 방식이다. 이를 통해 Neural Engine은 MAC 연산에만 집중할 수 있다
- 연산 명령어
 - Neural Engine Cluster(이미지 참고)가 SRAM에 준비된 데이터를 사용해 MAC 연산을 수행하도록 한다
- 의존성 스케줄링
 - 컴파일러는 연산 명령어가 필요한 데이터 이동 command 완료된 이후 실행되도록 command dependency(의존성)를 생성한다
 - 필요한 데이터가 SRAM에 모두 준비된 이후에 연산이 실행되도록 보장한다



Overlapping Execution

